GC1

Geschichte

Chemieinformatik: is the application of informatics methods to solve chemical problems

↳ Chemistry deals with compounds, their properties and their transformation

Experiment → Daten → Modelle → Vorhersage

Konzept des induktiven Lernens:
↳ erstellen Modell → Eigenschaften vorhersagen

- Automatisierung: Durchführung vieler Experimente
- Informationstechnologie: Grundlage zur Speicherung, Manipulation und Analyse großer Datenmengen
- Chemieinformatik: ermöglicht die Nutzung induktiven Lernens in der Chemie im Zeitalter der Informationstechnologie

- Frage: Struktur Eigenschaftsbeziehungen
- Computer-Aided Molecular Design / Drug Design: Entwicklg Verbindgen
- Computational Chemistry: Modellierg & Vorhersage
- Bioinformatik: Analyse Gene

GCI

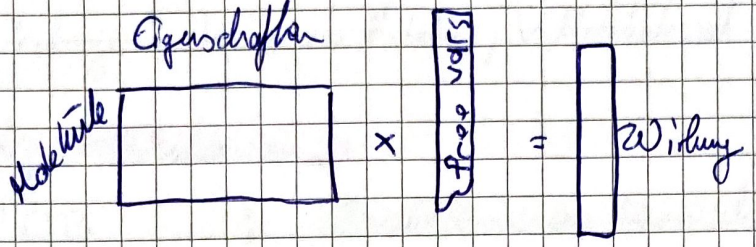
Bereich Bioinformatik CI

Target Discovery ⇒ Lead Generation ⇒ Lead Optimization ...
Bioinformatics Cheminformatics

Geschichte

1960+: Graphentheorie auf Chemie bezogen

1962: QSAR-Studie: Beziehung zw. Molekülstruktur & Wirkung



GCI

Datenbanken & Suchmaschinen

Chemiedatenbanken:

→ Speichern Moleküle mit:

- → biologischen Effekt
- → Nebeneffekte
- → physikalische Eigenschaften
- → Tox-Parameter
- → Infos zur Synthese
- → Patenteninformation

e.g.: Anfragen: Infos zu Molekülen, Moleküle mit best. Eigenschaften

Wichtige Datenbanken

PDB	:	Strukturen von Biomakromolekülen, Proteine
CHEMBL	:	Moleküle + biologische Profile
ZINC	:	Kommerziell erhältliche Substanzen
PUBCHEM	:	Öffentliche Screening-Daten, kleine Moleküle
BRENDA	:	Enzyme
DRUGBANK	:	Wirkstoffe
SureCHEMBL	:	Moleküle + Patente
CSD	:	Kristallstrukturen

Suchmaschinen: Chemspider, eMolecules

Scifinder: Alles zu Chemie

MDDR: Wirkstoff-Moleküle

WDI: Wirkstoff-Moleküle

MDDR

- Datenbank mit Strukturinformationen
- MDL Drug Data Report
- MDL ISIS: Beschreibg therapeutische Funktion
- Suchmethoden: Textsuche, Substruktursuche in Molekülen, 3D Pharmakophorsuche

WDI und DNP

- World Drug Index: WDI
  - ↳ vermarktete und in Entwicklung befindliche Wirkstoffe
  - ↳ Suchmethoden: Textsuche, Substruktursuche in den Molekülen
- Dictionary of Natural Products: DNP
  - ↳ wie MDDR & WDI bezogen auf Naturstoffe

CAS → Einträge Journals

Chemical Abstract Service: Einführung CAS-Nummern

↳ SciFinder: Schnittstelle zu CAS & MEDLINE

↳ Suchmaschine

CSD → Kristallstrukturen

Cambridge Structural Database:

↳ Datenbank für Röntgenstrukturanalysen und metallorganische Verbindungen

↳ wichtigste Informationsquelle in Bezug auf Molekül- und Kristallstrukturen

↳ Atomkoordinaten

GCI

PDB

Protein Datenbank → PDB-Identifier

Strukturdaten von Biomakromolekülen.

↳ Ligand Expo: Teil der PDB und enthält Zusatzinformationen für kleine Moleküle in der PDB.

Pub Chem

Datenbank der NIH-Roadmap Initiative Molecular Libraries and Imaging Implementation

→ Strukturen & Infos zu kleinen Molekülen

Patent-Informationssysteme

Patente: Vielzahl verwandter Moleküle

↳ ähnliche auch Patent geschützt ↗ Struktursuche nicht möglich!

↳ Verschleiern von relevanten Informationen

Problem: Eine Substanz kann bereits in einem anderen Kontext oder unter einem anderen Namen patentiert sein

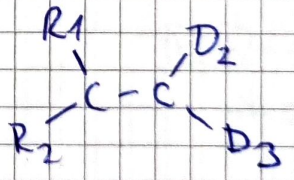
↳ Markush-Strukturen:

Grundgerüste + Listen von Gruppen

(+) Kompakte Beschreibung einer großen Anzahl verschiedener Moleküle

(-) Risiko: alle reaktionsfähigen Moleküle müssen die patentierte Eigenschaften aufweisen.

↳ z.B. SciFinder
Sure Chem
Excaptor



GCI

Workflow-Systeme

- Chemisch-informatische Datenprozessierung und -analyse ist gekennzeichnet durch:
  - große Datenmengen
  - sehr variable Prozesse

  => Skriptity

- Graphische Workflow Systeme
  - -> Pipeline Pilot
  - -> Knime

Pipeline Pilot

„rapidly create, test and publish scientific processes that automate the process of accessing, analyzing and reporting scientific data."

-> Baukastenprinzip.

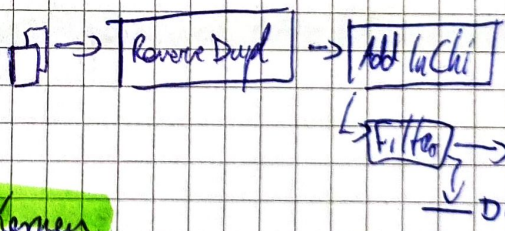
-> Datenmanagement und Datenmanipulation & Visualisierung

eg: Einlesen, Normalisieren, Entfernen Duplikate, Filter, Auswahl

Knime

-> open source graphisches Workflow-System

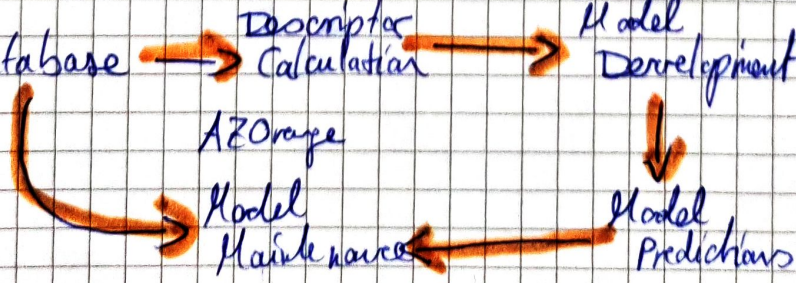
-> ähnlich wie Pipeline Pilot



~~AZ~~ AZOrange

- Vollständige Plattform für maschinelles Lernen
- effizient gestaltet in Workflows
- open source

- Orange: für Datenanalyse & Visualisierung, visuelles Programmieren (auch Python)
- QSAR analysen.



GCI - Ex

DrugBank

Warum findet man mehr als ein Ergebnis?
-> Entweder Enantiomere oder strukturell sehr ähnlich
-> Ähnlichkeit ist über einen Score angegeben.

PubChem

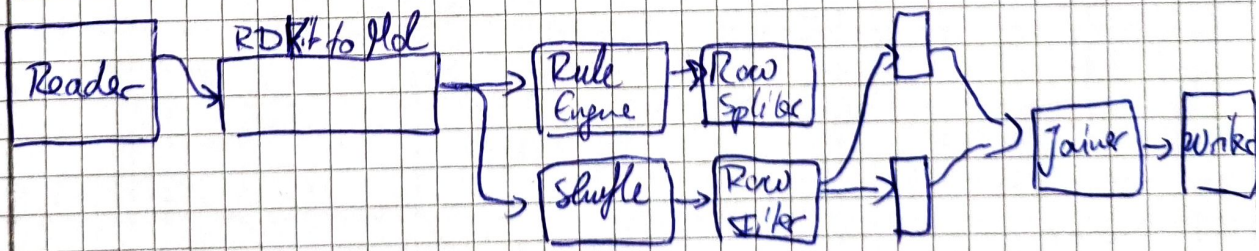
-> kleine Moleküle

PDB und BRENDA

PDB: 3D-Strukturen von Proteinen.
Nach welchen Kriterien eine Struktur auswählen, wenn man ein bestimmtes Protein sucht?
-> Resolution, Name of Source Organism, Taxonomy, Experimental Method

BRENDA
Warum gibt es unterschiedliche IC50-Werte für dieselben Inhibitoren?
-> Unterschiedliche Testbedingungen begünstigen oder hemmen die Bindung.

Knime



GC1

Molekülgraphen und File-Formate

2D-Struktur → markierter Graph

Anwendung: Identifizierung, Suche in DB's, Beschreibung von Syntheseregeln

Speicherung von Molekülen

Filformate eg: SD-Format, mol2-Format, PDB, SMILES
ASCII-basiert, zugrunde Formaldef's

- **PDB-Format:**
  - ASCII-basiert, fixed-format
  - template-basiert
  - 3D-Struktur, arm an chemischer Information

HETBE
HETATM
:

- **mol2-Format:**
  - ASCII-basiert, free format
  - Info: 3D-Struktur, reich an chemischer Information
  - Molekülbeschreibung besteht aus verschiedenen „Records"

<TRIPOS>
:

- **SD-Format:**
  - ASCII-basiert, fixed-format
  - erlaubt variablen Informationsgehalt
  - beliebig annotierbar

GdCh

SD-Format

- **Header:** Mol name, user, program, data, etc.
- **Counts Line:** #Atome, #Bindungen, x, Chiral-Flag, etc.
- **Atom Line:** Coordinates, Element, Mass Diff, Charge, etc
- **Bond Line:** Atom 1, atom 2, bond type, ...
- ~~Mol Properties~~
- **Data Block**
- **Molecule Separator** $$$$
- **Wichtige Parameter der Atom Line:**

x, y, z	Atomkoordinaten
a, a, a	Atomsymbol der Periodensystem
dd	Massendiff.
sss	Stereoflag
hhh	#Wasserstoffe + 1

- **Wichtige Parameter der Bond Line**

111, 222	Erstes & Zweites Atom
ttt	Bindungstyp
sss	Bond Stereo
xxx	—
rrr	Topology

GrC1

Kurzerklärung von chemischen File-Formaten

- Datenrepräsentation in File-Formaten liegt ein Modell zugrunde, dieses ist je nach Format unterschiedlich
- Informationsgehalt Formate

Daten \ Format	PDB	mol 2	sdf	SMILES
Element	X	X	X	X
Hybridisierung	–	X	–	X
Ladung	–	O	O	X
Wasserstoffe	O	X	X / I	X / I
Kovalente Bindung	O	X	X	X
Bindungstyp	O	X	X	X
Atomkoords	X	X	X	–

X: enthalten, –: nicht enthalten, O: optional, I: implizit

Kurzerklärung: fehlende Informationen müssen abgeleitet werden.

Daumis Valence-State Modell

Valence State: # der Einfach, Doppelbindg, die Formalladg & # H an einem Atom

Atom-Type: Geometrieinformationen & Delokalisierungsverhalten

1. Element → 2. Valence State → 3. Atom Type

GaCl + Cl2

Mona Basic

Verarbeitungstool für Moleküle -> Mona.

Große Mengen von Molekülen zu navigieren, Filtern, Teilmengen bilden, Kurzfassung finden.

- In welcher Hinsicht können sich zwei Instanzen unterscheiden?
  Molecule Properties: Masse, Größter Ring, # Atome
  ↳ LogP
- Warum könnten sie nach Filtern Moleküle, die nach Filterkriterien ausgefiltert sein müssten?
  Lipinski Rule erlauben einen Verstoß gegen eine der Eigenschaften.
- Unterschiede Dateiformate, sdf, mol2, smi
  sdf: Atome (C & H) + Akzeptoren, Donoren, etc., Annotationen
  mol2: Zu jedem Molekül Name, Größe, Atome, Bonds & Koords
  smi: Sehr kompaktes Dateiformat, Lineares Format
- Moe: .moe format
  ↳ Datum ↳ View Orientation
  ↳ # Moleküle ↳ Bindungen
  ↳ Eigenschaften Moleküle: Element, Koords
  ↳ Eigenschaften Reste
  ↳ Formatierung der Ribbons

GrC1 - G2

Ligand Exp

- Unterschied total Molecules und gelaxierte Molecules?
  ↳ Duplikate werden entfernt

DrugBank

- Warum hatten Moleküle gleichzeitig approved & withdrawn sein?
  ↳ Wenn ein Wirkstoff zugelassen wurde und später wieder zurückgezogen wurde.

Prozentual mehr Moleküle erfüllen die Lipinski Rule of Five in der DrugBank gegenüber Drug Central Database.

GC1

Lineare Molekülrepräsentation

Lineare Molekülrepräsentation:
- → Darstellung eines Moleküls durch eine Zeichenkette
- → Topologie des Moleküls

Ziel:
- einfaches, effizientes Arbeiten mit Molekülen
- kompakte Darstellung
- schnelle Identifikation von Molekülen

eg: WLN, SMILES, SLN, InChI

SMILES

einfach zu konstruieren.

Grundregeln:
1) Atome werden durch Element-Symbol dargestellt
2) Wasserstoffe werden implizit beschrieben (frei Valenz gesättigt)
3) Kovalent gebundene Atome stehen benachbart in der Zeichenkette
4) Doppel-B: =, Dreifach-B: #
5) Verzweigungen durch Klammern dargestellt
6) Ringe durch Zahlenpaare

- Atome:

Isotope ↑ | [H X] # Wasserstoffatome | Ladung

atom: [<mass> symbol <chiral><hcount><charge>]

- [ ] darf für organische Subsetze weggelassen werden: B, C, N, O, P, S, F, Cl, Br, I

eg: H_2O: O , OH^-: [OH-] , Fe^{+2}: [Fe+2]

- Bindung:

Ein | Doppel | Drei | aromatisch

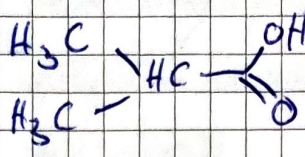
bond: <empty> | - | = | # | :

eg: $H_2C{=}O$ $C{=}O$

GCI

SMILES

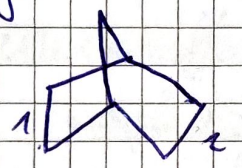
- Verzweigung:
  -> einzelne Zweige ( ) , kann gerückt/geschachtelt werden

eg:  -> CC(C)C(=O)O

- Unterbrechung:
  ; : beschreibt eine Unterbrechung

- Ringe:
  Matchendes Zahlenpaar
  -> Entfernt Bindungen, bis ein Baumstruktur entsteht, -> Zahlenpaare

eg:

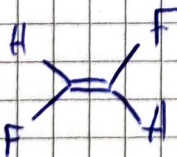


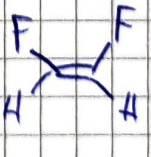
C(C1)(C2)CC(C1)C2

C=1CCCCC1

- cis/trans-Isomerie:
  /\ k /\

eg:

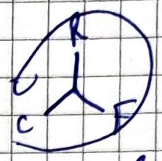


F/C=C/F ,  F\C=C/F

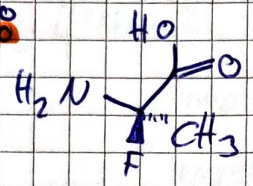
GC I

SMILES

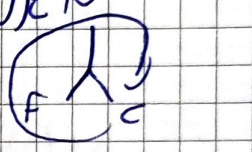
- Chiralität an tetraedrischen Kohlenstoff:
  - ↳ lokale Chiralität:
    erste Bindung im String definiert die Bildrichtung, die folgenden Gruppen haben als Ordnung die Reihenfolge ihres Auftretens.
    - → @
    - → @@



e.g.:



- → N[C@](C)(F)C(=O)O
- → N[C@@](F)(C)C(=O)O
- → C(=O)(O)[C@@](F)(N)C



- Wasserstoffe
  - Hybridisierung, C: 4, N: 3, O: 2 ...
  - [ ] → 0 H's
  - [H] → 1 H
- Aromatizität
  - Verwendung Kleinbuchstaben (c, o, n, s)
  - Verwendung des „:" für aromatische Bindung

GC1 - Ex3

3-1 SMILES

1)

a) Gold : [Au]
b) Kupfer (II) : [Cu+2]
c) 13C : [13C]
d) Methan : C
e) Ammonium : [NH4+]

2) Implizit / Explizit Wasserstoffatome:

a) Methan:

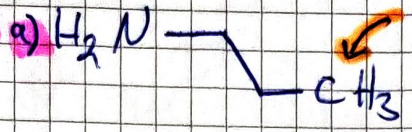
implizit: C → freie Valenz wird gesättigt
explizit: [CH4]
C([H])([H])([H])[H]

b) C1=CC=C(C=C1)C(CC(=O)O)NC(=N)N

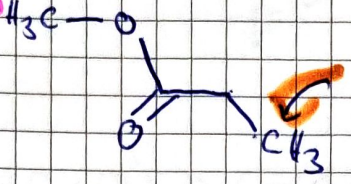
1) Neutrale Form
2) Wasserstoffe an C implizit
3) Wasserstoffe an O/N explizit

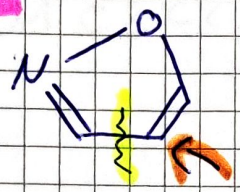
C1=CC=C(C=C1)C(CC(=O)[OH])[NH]C(=[NH])[NH2]

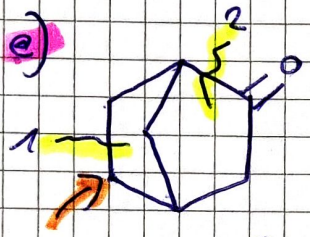
3-1

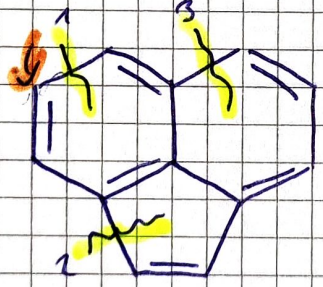
3) a)  : CCCN

b) : CCC=O

c)  : CCC(=O)OC

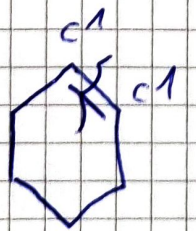
d)  : C1=COC=C1

e)  : C1C(CC2C1)CC2=O

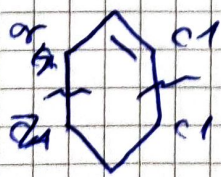
f)  : C1=CC2=C(C3=C1)C(C=C2)=CC=C3

<=>

C1CC2C(C3C1)C(C2)CCC3



C=1CCCCC1

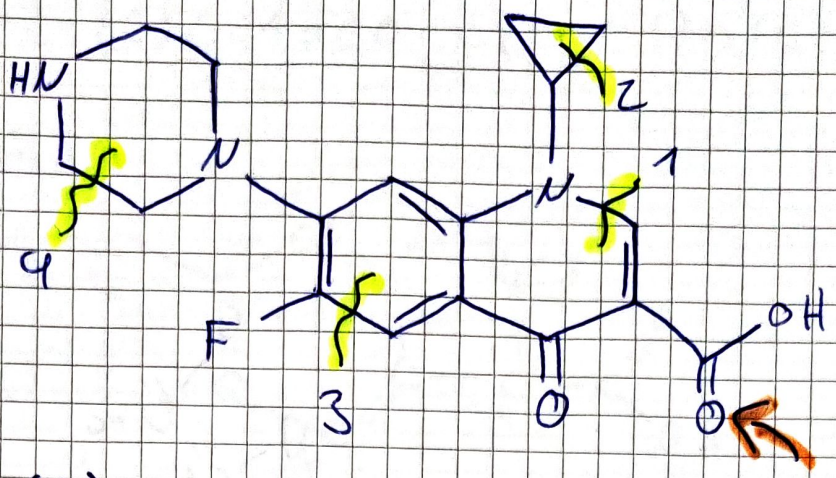


C1CCCCC=C1

GACI

3-1)

3) g)



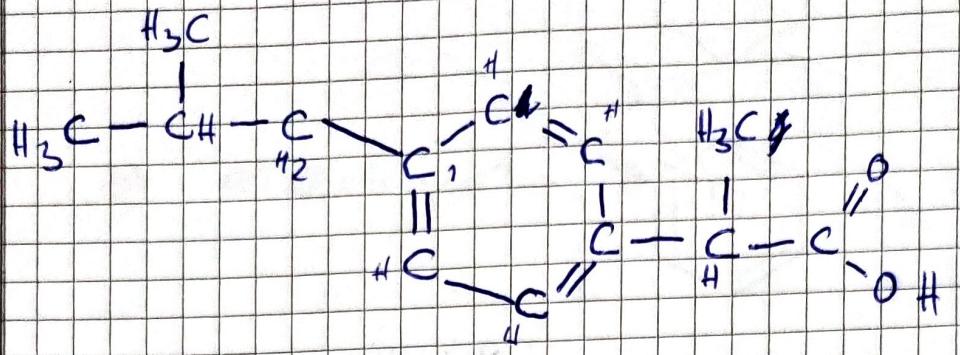
O=C(O)C(=C1)C(=O)C(=C3)C
(N1C2CCC2)=CC(=C3F)N(C4)CCNC4

GC1

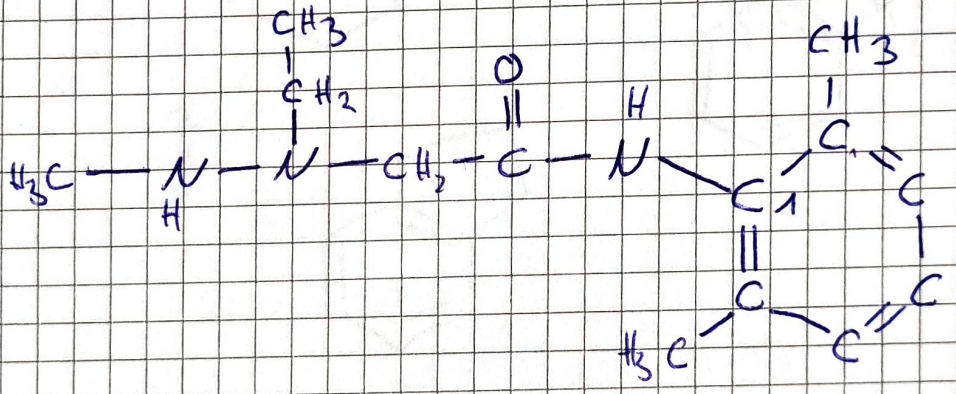
5 → 10

4)

a) CC(C)CC1=CC=C(C=C1)C(C)C(O)=O



b) CNN(CC)CC(=O)NC1=C(C)C=CC=C1C

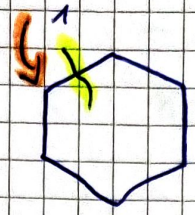


GC1

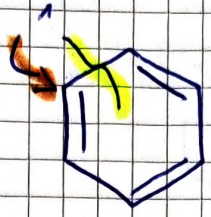
3-1)

5)

a) C1CCCCCC1

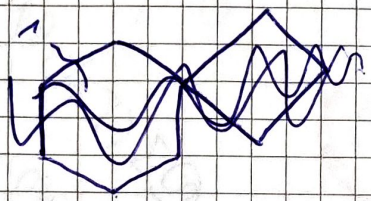


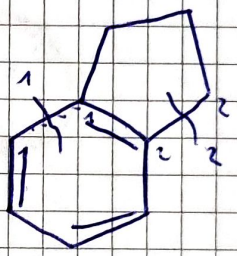
b) c1ccccc1

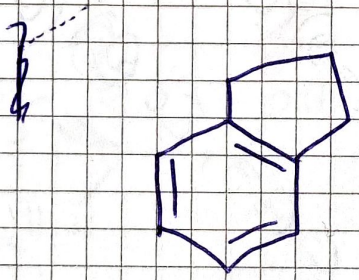


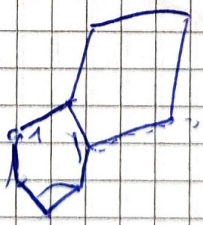
c) valider SMILES

c1ccc2c1CCC2





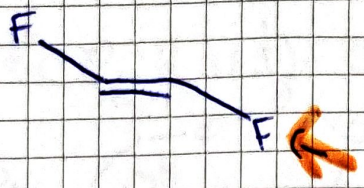




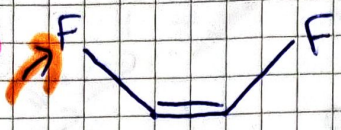
GC1

(5-2)

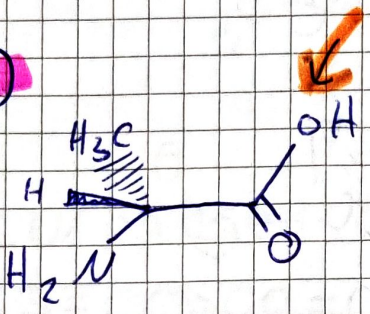
1)

a) 

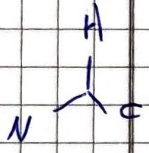
: F/C=C/F

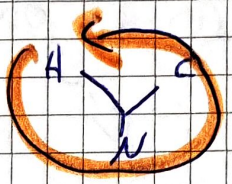
b) 

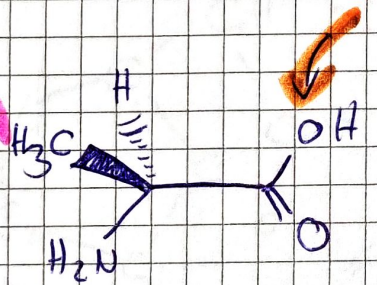
: F\C=C/F

c) 

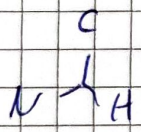
: OC(=O)[C@](C)(N)[H]

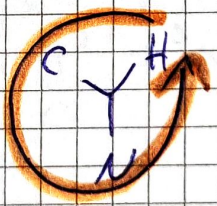




d) 

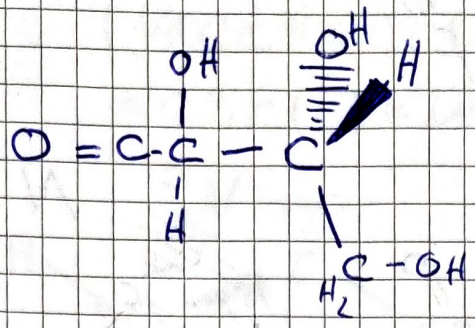
: OC(=O)[C@](N)(C)[H]

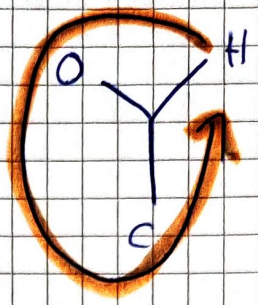




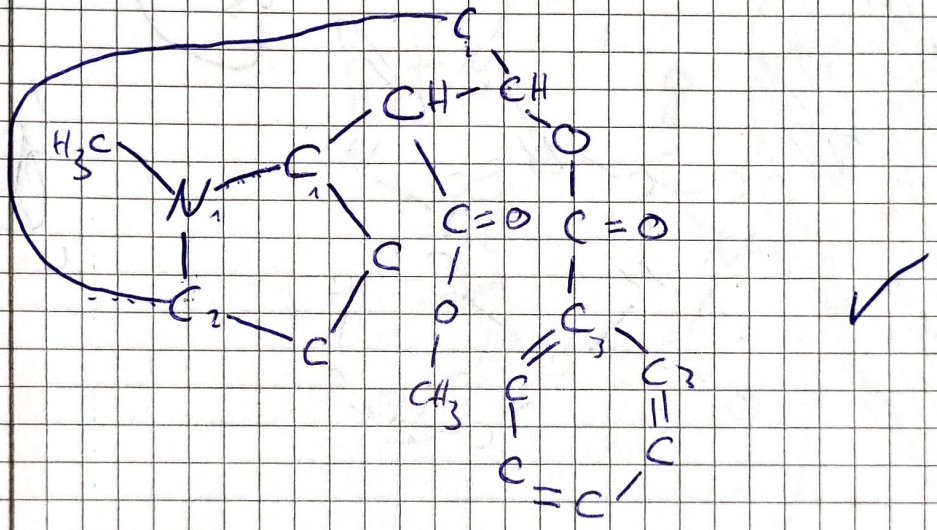
(3-2)

2) a) O=C[C@H](O)[C@@H](O)CO

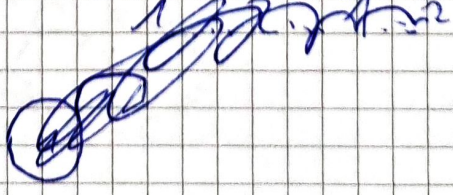




b) CN1C2CCC1[C@H]([C@H](C2)OC(=O)C3=CC=CC=C3)C(=O)OC



1...2...2...1

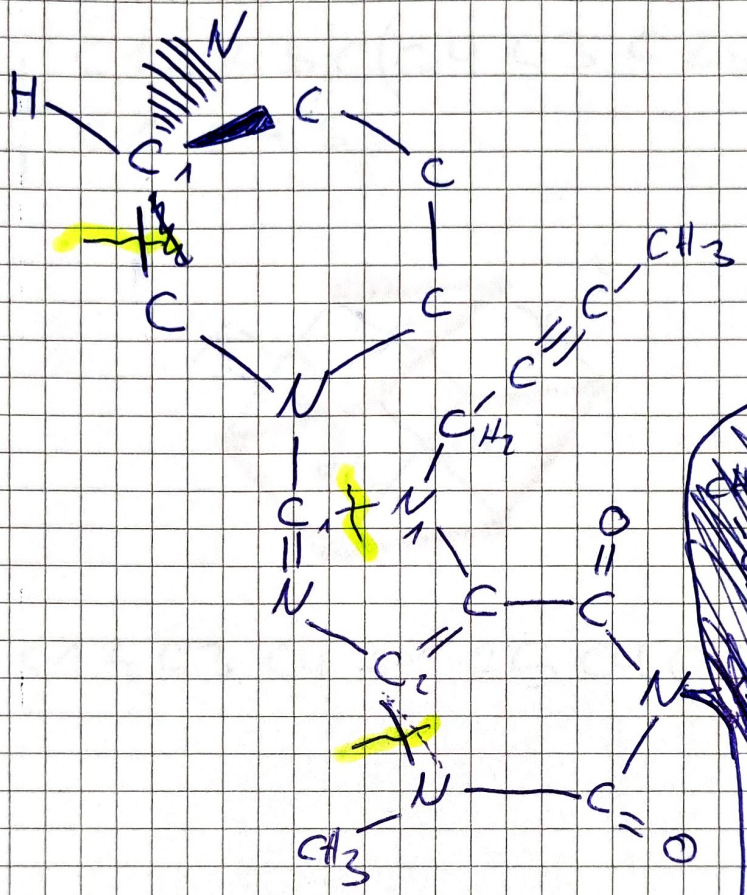


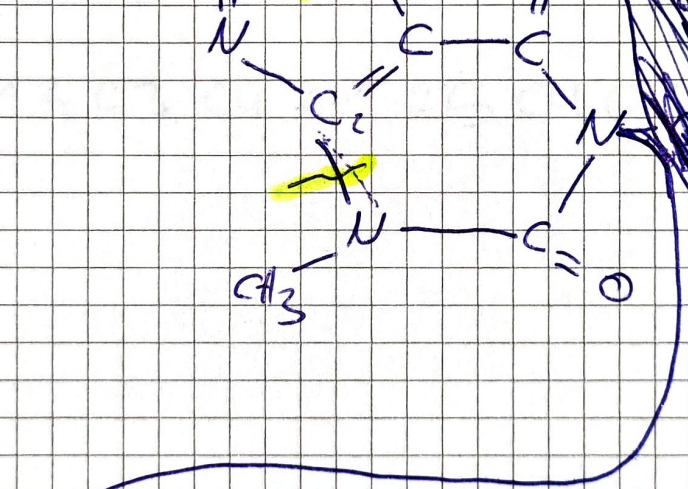
QCM

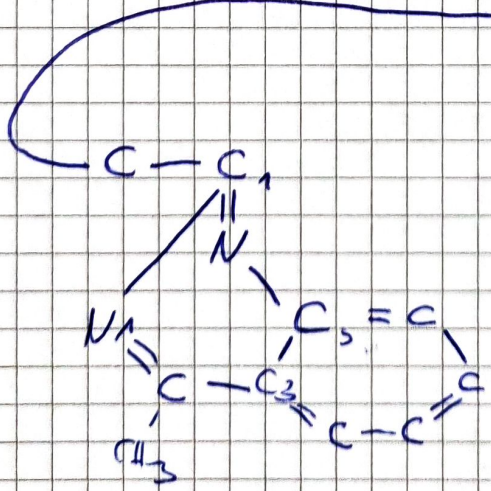
S-2

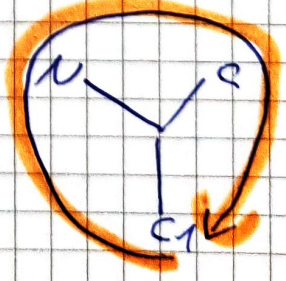
2)

c) [H][C@@]1(N)CCCN(C1)C1=NC2=C(N1CC#CC)C(=O)N(CC1=NC3=CC=CC=C3C(C)=N1)C(=O)N2C





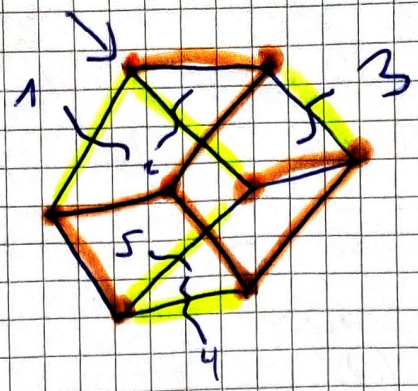




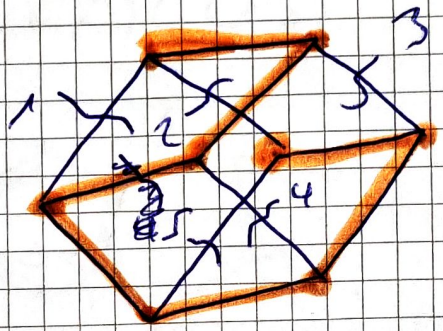
GC1

(3-2)

3)



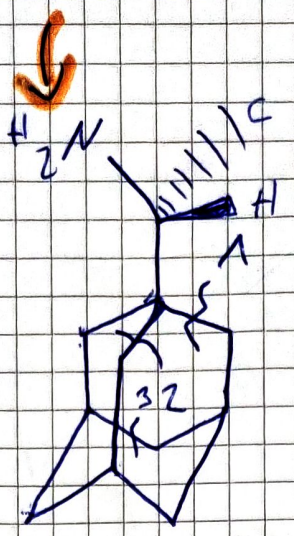
C12 C3 C(C4 C3 C25)
C1 C4 5



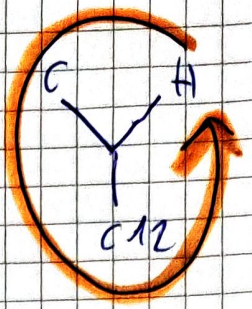
C12 C3 C4 C1 C5 C4 C3 C25

QCM

S-2)



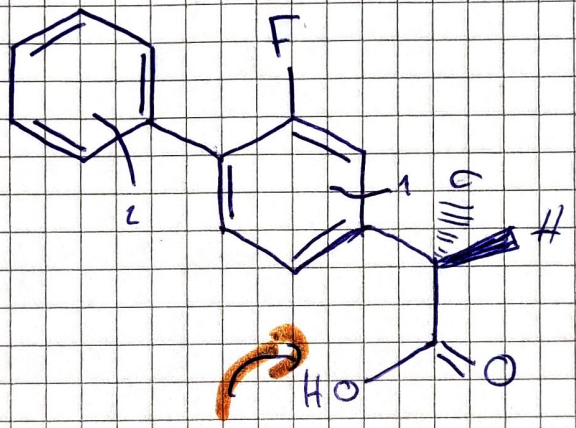
3)

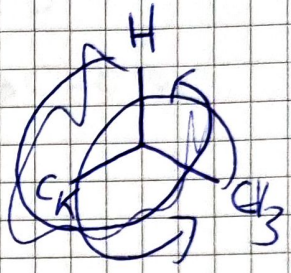


5)

N[C@](C)(C(=O)O)C12CC3CC(CC(C1)C3)C2

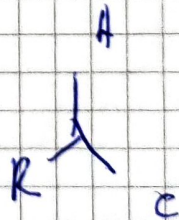
c)





O=C(O)[C@](N)(C)[H]

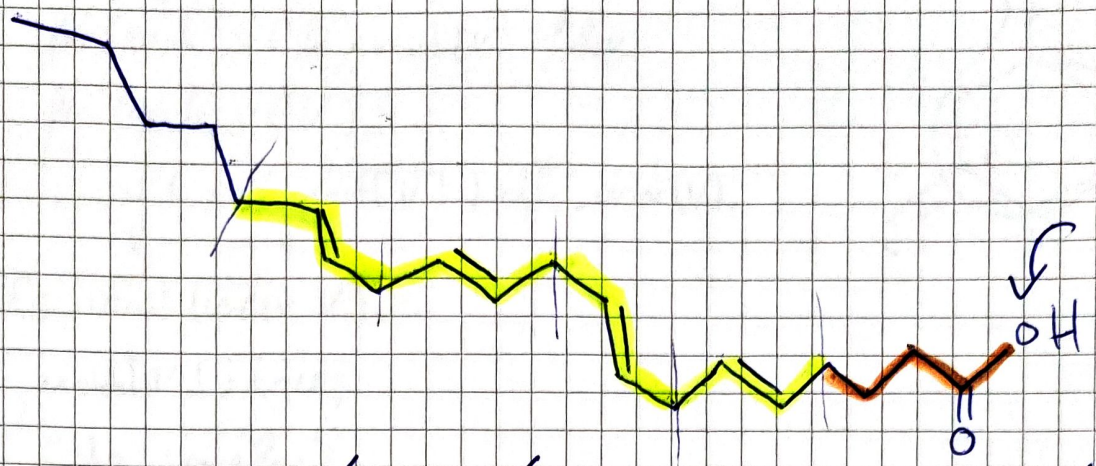
C1=CC=C(C(F)=C1)C2=CC=CC=C2



GACI

3-2)

3) d)



OC(=O)CCCC/C=C\C/C=C\C/C=C\CCCCC

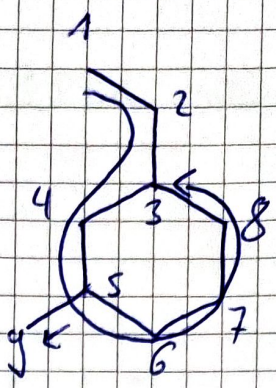
GCI

SMILES

Erfi Tidge

Tiefensuche (DFS)

```
DFS(graph G=(V,E))
    for(each v∈V) visit[v]=false;
    for(each v∈V)
        if( not visit[v] ) dfs_visit(v);

dfs_visit(vertex v)
    visit[v]=true;
    <do something>
    for(each u∈v.adj)
        if(not visit[v]) dfs_visit(u);
```



Generierung von SMILES

Problem: gegeben: Molekülgraph gesucht: SMILES des Moleküls

Alg: (2 DFS-Durchläufe)

I) Bestimme & Nummeriere Ringschlüsse $r: E \to \mathbb{N}_0$

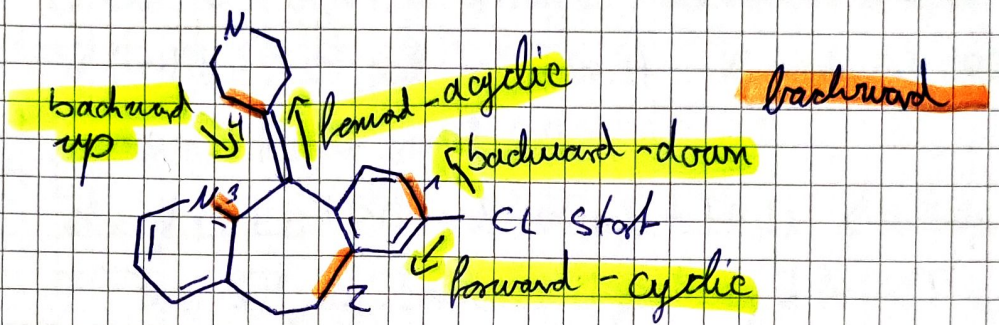
```
MARK-RING-EDGES(atom v, atom p, bool visit[G.V], int r[G.E], nr)
    visit[v]=true
    for(each e=(v,w)∈v.adj with w≠p)
        if(visit[w]) nr++; r[e]=nr;
        else r[e]=0; MARK-RING-EDGES(w,v,visit,r,nr)

mark_ring_edges(v, NIL, vist, r, nr), visit[.]=false, nr=0
```

(Wenn ein Nachbar schon mal besucht wurde außer parent dann wurde ein Ring gefunden) ⇐

GrCl

SLIDES

- Nach I°
  - -> Kanten sind als DFS-Baumkante (forward) oder DFS-Rückkante (backward) markiert
  - => Rückkanten sind numeriert
  - -> Baumkanten -> azyklisch / zyklisch
  - -> Eine Rückkante (z,v) verläuft von z aus gesehen im Baum aufwärts (up) von v aus gesehen im Baum abwärts (down)



Generierung von SMILES

III) Generiere SMILES

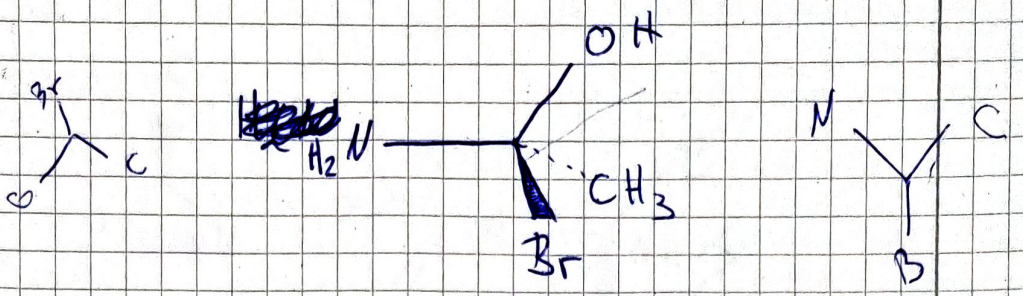
```
GENERATE-SMILES (atom a, bond toa, int r[E], string &S)
  if ( toa ≠ NIL ) s = s || print_bond (toa)
  s = s || print_atom (a)
  visit [a] = true
  nof_fwd = 0
  for (each b = (a,v) ∈ a.adj)        // erster Durchlauf
      if ( r[b] > 0 and visit [v] )    // backward-up Bindg
          s = s || print_ringid (r[b])
      else if (r[b] > 0 and not visit [v])   // backward-down Bindg
          s = s || print_bond (b) || print_ringid (r[b])
      else if (b ≠ toa)
          nof_fwd ~~++~~ ++            // Bestimme Degree
  for (each b = (a,v) ∈ a.adj)        // zweiter Durchlauf
      if (r[b] == 0 and b ≠ toa)       // forward Bindg
          if (nof_fwd > 1 ) s = s || "("
          GENERATE-SMILES (v, b, r, S)
          if (nof_fwd > 1) s = s || ")"
          nof_fwd --
```

⇒ Keine Dynamischen Chiralitäten

⇒ Keine Tautomere

GCL - Gx4

(4-1)



1) a) O[C@@](C)(N)Br ✗

b) C[C@@](O)(Br)N ✗

c) O[C@](Br)(C)N ✓

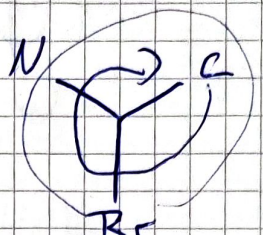
d) C[C@@](Br)(O)N ✓

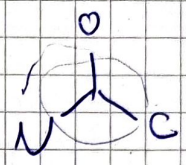
e) Br[C@](C)(O)N ✓

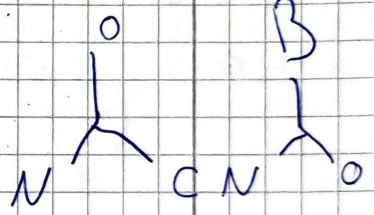
f) Br[C@@](N)(O)C ✓

g) Br[C@](C)(N)O ✗

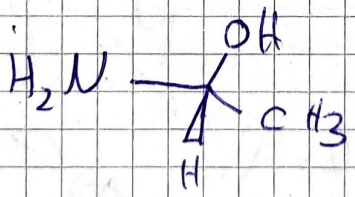
h) C[C@](Br)(N)O ✓

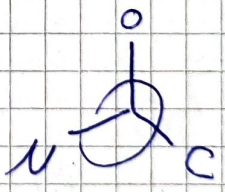






3)





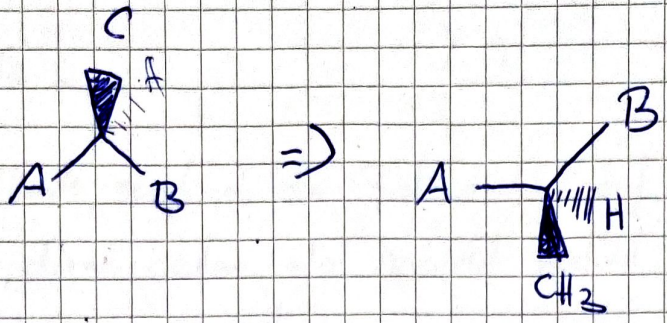
[H][C@](N)(C)O

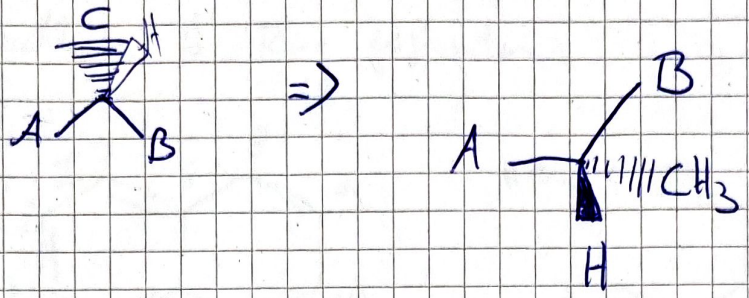
GC

4 - D

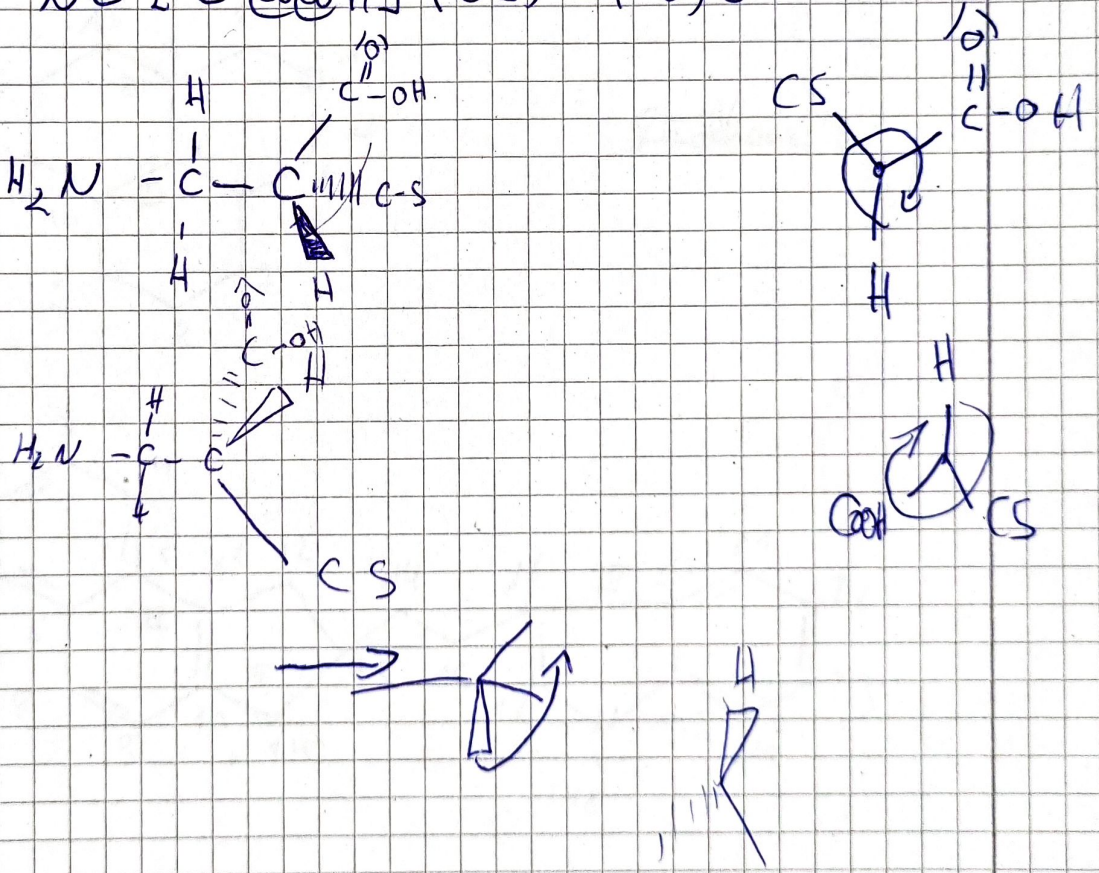
2)

a)



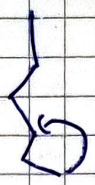


b) NC[C@@H](CS)C(=O)O



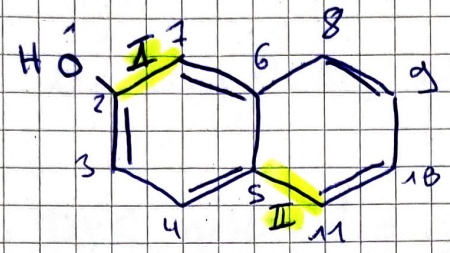
(4-2)

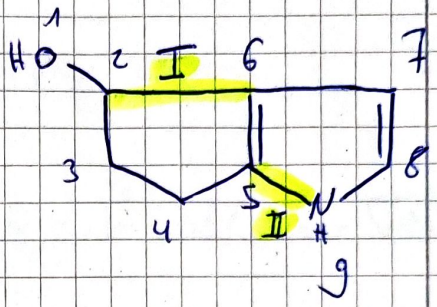
a) Rückkante.

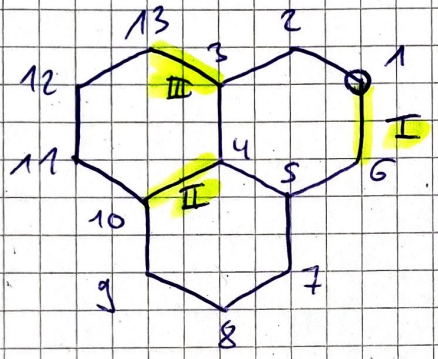


Eine Rückkante zeigt in der Tiefensuche auf einen Knoten, dieser schon als visited markiert wurde und nicht der Parent ist.
↳ Rückkante ≙ Ring gefunden

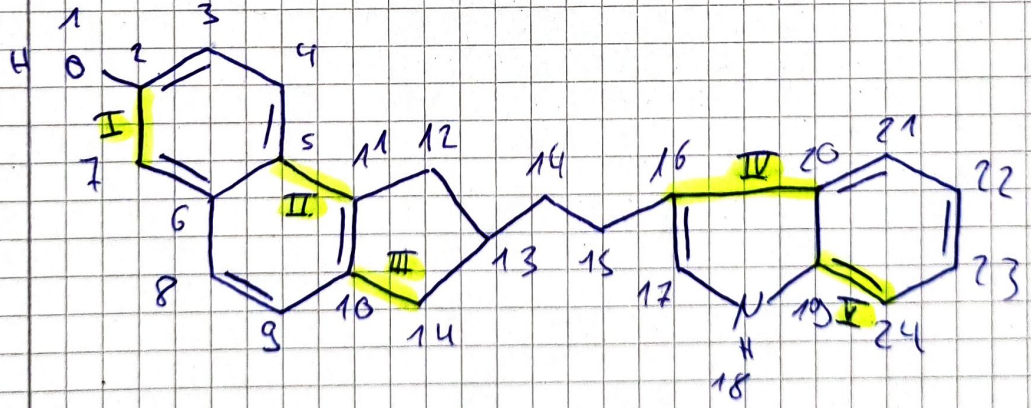
b)







Rückkante



GC1

(4-2)

3) Welche nicht durch DFS?

1) 0 C1 CC2 C (C C C C2) CC1 ✓

2) oC(C1) C2 C(C1) NCC2 ✗

3) N1C2C(CC1) CCCC2 ✓

4) N1 CC(N2)C(NC2)NC1 ✗

↳ DFS geht evt ganze Tiefe durch nicht nur Teil!

QC1

4-5

4) Generiere SMILES mit wiederholenden Ring IDs

```
GEN-SMILES(atom a, bond toa, int r[E], string& s,
                                  int max_id=0, stack={})

if (toa ≠ NIL) s = s // print_bond(toa)
s = s // print_atom(a)

vis.it[a] = true
nof_fwd = 0

for (each b = (a,v) ∈ a.adj)                    // Schleife
    if (r[b] > 0 ad vis[v])
        s = s // print_rnr_id(r[b], max_id, stack, false)
    else if (r[b] > 0 kk not vis[v])            // Öffnen
        s = s // print_bond(b) // print_rnr_id(r[b], max_id, stack, true)
    else if (b ≠ toa)
        nof_fwd ++

for (each b = (a,v) ∈ a.adj)
    if(H[b] == 0 ad b ≠ toa)
        if (nof_fwd > 1)  s = s // "("
        GEN-SMILES(v, b, r, s, max_id, stack)
        if (nof_fwd > 1) s = s // ")"
        nof_fwd --
```

GAC1

(4-3)

4)

```
print_ring_id(int r, int max_id, stack stack, bool opened)
```

~~if not opened:~~
~~   r_id = stack.pop()~~
~~   print(r_id)~~
~~else:~~

```
if opened:
    if stack.empty():
        max_id++
        stack.push(max_id)
    r = stack.pop()
    print(r)
else:
    stack.push(r)
    print(r)
```

Idee: Nehme aus Stack heraus und wenn kein Element vorhanden dann zähle hoch & übergebe Ring Id mit zurück vom Stack